

Round-Robin Adversarial Review as a Verification Methodology

*What 24 rounds of independent review found in a
financial pricing library, and why the fixes kept needing
fixing*

DRAFT V1

24

INDEPENDENT
REVIEW ROUNDS

*Codex, Fable,
Claude Opus*

507

TESTS PASSING
0 through round 22

3

FIXES THAT
INTRODUCED A
NEW DEFECT
rounds 7, 22, 24

6,808

LINES, ONE FILE
*8 instrument
classes*

Oluwaseyi Awoga · oawoga@engineering.upenn.edu · `duration_engine.py` · 6,808
lines · 8 instrument classes · August 2026

ABSTRACT

We report on a 24-round independent-review process applied to a single-file financial pricing library covering bonds, floating-rate notes, interest-rate swaps, bootstrapped yield curves, swaptions, and forward-rate agreements. Each round was conducted by a reviewer with no memory of prior rounds' own claims, briefed to re-derive every finding from the code directly rather than accept a prior round's word for what was already correct. Across 24 rounds we found a long tail of genuine, previously undetected defects with no sign of a plateau, including three distinct cases in which a round's own fix, once independently re-reviewed, turned out to have introduced a new defect of a similar shape to the one it had just corrected. We describe the round structure as a reusable process, report what kinds of defects this method is and is not good at catching, and argue that its blind spot (sys-

tematic exploration of an input space, as opposed to spotting a specific bad case) is best closed by property-based testing as a complement, not a replacement.

1. Introduction

Financial pricing and risk software has a failure mode that ordinary testing under-serves: the silent wrong answer. A pricing function that raises an exception on bad input announces its own failure. A pricing function that returns a plausible-looking number computed from the wrong formula announces nothing at all. In a library whose whole purpose is producing numbers a reader will trust (a modified duration, a swap's par rate, a swaption's present value), this second failure mode is the one that matters most, and it is exactly the one unit tests written by the same author who wrote the bug tend to miss, because the author's mental model of "correct" and the code's actual behavior share the same blind spot.

This article reports on an attempt to address that blind spot with a specific process: round-robin independent review. Over 24 rounds, a financial derivatives pricing library (`duration_engine.py`, a single Python file that grew from roughly 1,500 to 6,808 lines over the course of the project) was repeatedly handed to a reviewer, sometimes an external tool (Codex), sometimes a large language model briefed to stand in for one (Claude Opus, Fable) when the primary tool was unavailable, with an explicit instruction to find what prior rounds had missed, not to re-confirm what they had already found. Every finding, from either side, was independently re-derived by direct reproduction against the running code before being accepted or fixed; nothing was taken on a reviewer's word, including this project's own.

The central empirical question this article asks is simple: does this process keep finding genuine defects past the point where a reasonable prior would expect diminishing returns, and if so, what do the defects it keeps finding look like? The answer, from this one codebase, is that 24 rounds showed no clear plateau, and that the most concerning class of finding in the later rounds was not a defect in old, previously unexamined code, but a defect freshly introduced by the *previous* round's own fix. We call this pattern regression-in-a-fix and treat it as the article's central finding, alongside two smaller but portable lessons the process surfaced along the way: that an inherited or derived configuration value should be tested by its effect, not by the setting that produced it, and that a plausibility check calibrated against one market regime can be simultaneously too strict and too loose once applied to another.

We are explicit about what this article is not. It is not a benchmark comparing review tools, since which reviewer was used in a given round was driven by tool availability rather than a designed comparison. It is not a claim that this process generalizes to other codebases, domains, or team structures, since it reflects a single project with a single author orchestrating the rounds. And it is not, on its own, a certificate of financial correctness, a point we return to directly in Section 7, because it is the sharpest limitation of everything this article reports.

2. Background

Three traditional approaches bear on the problem this article addresses. Golden-file regression testing, pinning a set of known-correct output figures and re-checking them after every change, catches the case where a change breaks something that used to work, but by construction says nothing about a defect that was present in the golden figures from the start; this project used it throughout (see Section 5) and it never once found a genuine defect on its own. Property-based or generative testing, asserting an invariant and letting a framework search the input space for a counter-example rather than hand-picking one, is well suited to exactly the kind of defect this project's independent reviewers tended to find (a specific, non-obvious combination of parameters that breaks an assumption), but this project did not adopt it until very late, and only reactively, a point taken up in Section 6. Human or LLM-assisted code review is the closest relative of the process described here, but almost always as a single pass: one reviewer, one round, done. What is less common, and what this article is really about, is treating review as a repeatable process applied to the *same* evolving codebase, where each round's reviewer has no access to what the project itself believes is already settled, and where the project's own discipline is to verify every claim before acting on it, symmetrically in both directions.

This article does not attempt a formal-methods framing (no theorem provers, no SMT-based verification were used) and does not claim the review rounds constitute an adversarial security audit: every defect reported here is an "honest mistake" class finding, wrong on a legitimate input, not a vulnerability reachable only through malicious input construction.

3. Methodology

System selection. `duration_engine.py` was chosen not because it is representative of production financial software in every respect (it is a single file, with no external dependencies beyond `numpy` / `scipy` / `pandas`, and it is explicitly labeled a teaching library rather than a production pricing system), but because it grew under realistic conditions: new instrument features were added round over round, not held static, so the review process had to contend with a moving target rather than a fixed, frozen artifact, a property we return to as a threat to validity in Section 7.

Round structure. Each round followed the same shape. A reviewer, an external tool when available or an LLM agent explicitly briefed to stand in for one otherwise, examined the current state of the file, sometimes the whole file from scratch (rounds 4, 5, 21), sometimes specifically the diff since the previous round (rounds 2, 3, 6, 7, 22, 24), with no access to this project's own prior claims about what was already correct. Every finding it returned was independently re-derived by direct reproduction against the running code, not accepted because the reviewer's explanation sounded right, before any fix was written. Confirmed defects were fixed, and the fix was itself re-verified against the reviewer's exact reproduction, plus a full re-run of every previously published example, before being written up. Documentation (`README.md`'s per-round section, a `CODEX_REQUEST_Duration_Engine_Review_RoundN.md` file summarizing the round

and posing open questions back to the reviewer) was written last, describing only what had actually been verified.

Reviewer rotation. Codex was the primary external reviewer for most rounds. Claude Opus and, for round 23, Fable stood in when Codex access was unavailable, and were explicitly briefed as doing so; this was an availability decision, not a designed comparison, and this article makes no claim that any one of the three is systematically stronger or weaker at this task from this data.

What counts as a genuine defect, for this article's tallies. We count a finding as a genuine defect if a public, documented code path accepts an input or configuration that produces either (a) an exception where correct behavior would construct or price successfully, (b) a silently wrong numeric result relative to the documented or intended model, or (c) a silently wrong result relative to a previously established invariant of a sibling class, the clearest example being `InterestRateSwap` and `FloatingRateNote` disagreeing on byte-identical payment-lag inputs (round 24, discussed in Section 6). We explicitly exclude pure hygiene items (dead-code removal, naming consistency, docstring drift with no behavioral consequence) from this count, tallying them separately, to avoid inflating the headline defect number with items no reviewer would call a bug.

Regression-in-a-fix. A defect is tagged this way only when it did not exist before the specific round whose fix introduced it, and when that attribution is unambiguous from the record; this is a stricter bar than "found in code that was recently touched," and we discuss the looser reading, and why we didn't use it for the headline count, in Section 6.

4. The system under study

`duration_engine.py` covers eight instrument/utility classes: `FixedRateBond`, `FloatingRateNote`, `InterestRateSwap`, `ForwardRateAgreement`, `BootstrapCurve` (iterative stripping of par yields into spot rates), `Swaption` (Black-76), `DualCurveSwap` (OIS versus LIBOR discounting), and `SOFRTransitionCalibrator` (an illustrative curve-spread calibration exercise, explicitly documented as not the ARRC's official methodology). Supporting infrastructure includes day-count convention math (`DayCounter`, five conventions), a business-day calendar system, and shared payment-schedule generation used by every date-driven instrument.

One observation from 24 rounds is worth stating plainly before the Results section, because it shapes where a reviewer's time is best spent in a system of this kind: the large majority of genuine defects clustered in the cross-cutting infrastructure (business-day calendars, payment-lag inheritance, schedule generation, day-count boundary handling), not in the pricing formulas themselves. The Black-76 swaption formula, the discount-margin FRN model, and the bootstrap recursion were each reviewed and hardened, but the defects that kept recurring, including both regression-in-a-fix instances discussed in Section 6, were in the scheduling and calendar layer every instrument shares. A system built from a small number of pricing formulas sitting on top of a much larger date/calendar/schedule substrate should expect review effort to

pay off disproportionately in that substrate, not in the formulas a textbook would check first.

5. Results

Round-by-round record. Table 1 summarizes the 24 rounds. Six rounds (8, 9, 10, 12, 14, 18) produced no dedicated documentation section of their own: rounds 8–10 were folded into a single "direct implementation" write-up covering a larger round of feature work (schedules anchored to the effective date, per-leg day counters) rather than externally reviewed fixes, and rounds 12, 14, and 18 were audit-only passes that confirmed a prior round's work without finding anything new to fix. We list them in the table for completeness but exclude them from the defect tallies below, since there is nothing to tally.

Round	Source	New/changed defects found	Regression-in-a-fix?	Notable
1	Codex	6+ (schedule anchoring, Black-76 formula, OIS recursion, FRN index duration, day-count/ bootstrap)	n/a (first draft)	Modified Duration corrected 6.0038 → 6.5554
2	Codex	4 (FRN period conflation, macaulay closed form, index-rate interpolation, DualCurveSwap recursion)	loose reading only	
3	Codex	2 P0 + 3 P1 + smaller	loose reading only	Open question carried to round 4
4	Opus (stand-in)	4 new (annuity factor, silent option_type default, DM solver swallowing, annuity truncation)	no	Confirmed stale reviewed-attachment hypothesis

Round	Source	New/changed defects found	Regression-in-a-fix?	Notable
5	Opus (2nd instance)	6 (day-count propagation, falsy-yield ×5 sites, time-to-reset basis, market_price_type, exception safety, fixed_frequency)	no	Day-count Tier 1 scoped deliberately
6	Codex + Opus	5 (curve forward, DF-path unification, FRN reset-basis revert, bond solver swallowing, examples relabeled)	loose reading only	
7	Codex	5 (extrapolation forward, interpolation weighting, curve validation, allow_extrapolation, swap falsy-zero)	yes, strict count (round-6 extrapolation itself wrong)	
8-10	direct implementation	feature work, not review-driven	n/a	effective-date schedules, Group B swap cash flows
11	self-verified	0 defects (new-feature round)	n/a	overnight compounding + amortizing notionals
12	audit	0	n/a	no new findings
13	self-verified	2 (self-caught during verification)	n/a	payment lag + lookback
14	audit	0	n/a	no new findings

Round	Source	New/changed defects found	Regression-in-a-fix?	Notable
15	self-verified	1 (self-caught)	n/a	lockout + unequal-lag notional exchange
16	self-verified	1 (self-caught)	n/a	FRN calendar/lag/compounding
17	self-verified	1 (self-caught crash)	n/a	FRN lookback/lockout
18	audit	0	n/a	no new findings
19	Codex-style sweep	5 (FixedRateBond calendar drift, FRA validation, BootstrapCurve complex-number, Swaption NaN, DualCurveSwap validation)	no	Every remaining instrument class hardened once
20	audit	1	no	DualCurveSwap zero-boundary edge case
21	Opus (comprehensive)	20 (3 critical, 7 silent-failure, 10 cross-class)	no	First full whole-file review after 20 rounds
22	Opus (review of round 21)	2 confirmed flaws of 21 items checked	yes, strict count (dropped-coupon fix regression; arbitrage-check design flaw)	
23	Fable (comprehensive)	3 silent wrong-answer bugs + threshold rework	no	Started pytest suite, 475 tests

Round	Source	New/changed defects found	Regression-in-a-fix?	Notable
24	Opus (review of round 23)	2 confirmed flaws of 10 items checked	yes, strict count (payment_lag_calendar holidays regression)	Suite grew 475 → 507 across three follow-up passes

Strict count: 3 rounds (7, 22, 24), each an unambiguous case of a round's own fix introducing a new defect. Loose reading (rounds 2, 3, 6 included) discussed in Section 6.

Test-suite growth. No automated test suite existed through round 22; verification was hand-written markdown reproductions, re-run manually each round. Round 23 introduced `tests/` (475 tests); round 24 grew it to 507, including a new cash-date invariant sweep added specifically because two consecutive rounds (23 and 24) had found a real defect in payment-lag cash dates and the existing test for the relevant feature had asserted the mechanism rather than the invariant (Section 6).

Headline-figure stability. The library's published worked-example figures changed intentionally twice as real defects were fixed (bond Modified Duration 6.0038 → 6.5554 at round 1; swap Bloomberg modified duration -8.5303 → -8.3343 at round 9), then held byte-identical across every round from 9 through 24, sixteen consecutive rounds in which every fix, by construction, either touched a code path none of the six published examples exercise, or touched one without changing its output. This is a meaningful data point in its own right: it means the great majority of what 24 rounds of review found lived in code paths the headline examples never exercised at all, which is exactly what independent review of a growing feature surface should be expected to find, and exactly what a static regression suite pinned only to the headline examples would never catch.

Test-authoring bugs, counted honestly. Of round 24's sixteen initial test failures in the newly added cash-date invariant sweep, all sixteen turned out to be bugs in the test code itself (an index-misaligned zip, an unconditional business-day assertion that didn't account for `Calendar.NONE` meaning no adjustment, a maturity-date exemption that didn't account for positive payment lags) rather than in the library. We report this not as an embarrassment to omit but as a legitimate data point about verifying the verification: even a test suite written specifically to close a gap independent review had just found needed its own independent scrutiny before it could be trusted.

6. Discussion

Why fixes reintroduce defects of the same shape. The clearest instance is round 23 → 24. Round 23's fix gave `InterestRateSwap.payment_lag_calendar` a precedence order (the schedule-level `calendar` wins over the overnight-compounding calendar when set), and that precedence order was, on independent review, confirmed correct.

But the *implementation* of the winning branch reset the inherited-holidays collection to empty instead of carrying forward the caller's own `overnight_holidays`, so a caller-supplied market closure that was correctly honored before `calendar=Calendar.UK` was also supplied got silently dropped the moment it was. The round-23 fix's own review had focused on the calendar-precedence question (the part of the change that was actually being asked about) and did not independently re-verify the holidays side effect of the same edit. The round 21 → 22 case has the same shape: round 21's fix to a dropped-coupon bug was correct for a *valuation* date's schedule membership rule but wrong for a *contract-inception* date's, and the difference between the two only became visible once a reviewer checked the fix against an inception-anchored swap rather than the maturity-anchored bond the original defect report had used. In both cases, the fix was correct with respect to the specific counterexample that motivated it and incorrect with respect to a different case the same code path also has to handle, a pattern that suggests a fix's own review should deliberately probe cases *adjacent* to the one that motivated the fix, not just re-check the original counterexample.

Test the invariant, not the mechanism. Round 23's original test for the payment-lag calendar fix asserted `swap.payment_lag_calendar == Calendar.UK`, and passed, unchanged, on the round-23 configuration that we now know silently dropped the caller's holidays, because the calendar enum genuinely was `Calendar.UK`; only the holidays collection sitting behind it was wrong. The round-24 replacement test asserts the actual cash date and the full holidays set instead. The general lesson: for any inherited or derived configuration value, prefer asserting the *effect* (the date that got avoided, the number that got computed) over the *setting* that was supposed to produce it, because the setting can be exactly right while everything downstream of it is exactly wrong.

Absolute vs. scale-free plausibility checks. `BootstrapCurve`'s arbitrage check went through three redesigns across rounds 22, 23, and 24, and the final one illustrates a more general point about validating market data. An absolute cutoff on an implied forward rate (reject anything beyond, say, 50%/year) is really an implicit assumption about what a "normal" rate level looks like. Applied to a genuinely high-rate market (a flat 60%/year curve, which is internally perfectly consistent since its forward equals its level everywhere by construction, so it has zero shape inconsistency to detect), an absolute cutoff produces a false rejection driven entirely by the market's level, not its shape. The round-24 fix made the primary bound relative to the curve's own level (`max(absolute_floor, ratio × level)`), correctly separating "this market's rates are unusually high" from "this curve's shape is internally inconsistent," the two questions an absolute threshold necessarily conflates.

Quantifying the pattern, and why we didn't inflate it. Using the strict definition from Section 3, three of the roughly eighteen rounds that shipped a code fix (round 7, round 22, and round 24, all found via a round explicitly or effectively auditing the immediately preceding round's own change) introduced a defect of the same general shape as the one they were fixing. A looser reading that also counts round 6's revert of round 5's own `time_to_next_reset` basis choice, and the FRN period-0 index conflation,

tion that persisted, in slightly different forms, across rounds 1 through 3, would put the count closer to five or six. We report the strict count as the headline number specifically because the looser reading depends on judgment calls about causal attribution across rounds this project did not keep line-level diffs for (see Section 7); the strict count of three is defensible from the record without that judgment call, and is already, we think, the more interesting finding: a fix aimed narrowly at the case that motivated it has roughly a one-in-six chance, in this data, of getting something *else* wrong in the same code path.

What this method is not good at. Every regression-in-a-fix instance in this record was found by a human-directed reviewer examining a specific, hand-constructed case adjacent to the original counterexample, not by a systematic search of the input space. A property-based test asserting "for any (calendar, overnight_holidays, lag days, compounding) combination, every lagged cash date avoids every supplied closure" would very plausibly have caught the round-24 regression on the first attempt, rather than requiring a fourth round of review to surface it. This project only added that specific invariant sweep after an independent review recommended it: reactively, in response to a defect already found, rather than proactively as a standing check. This is, we think, the method's clearest limitation: independent review is well suited to finding a defect a specific author's blind spot missed, but it does not systematically explore an input space the way a generative test does, and even the hand-written parametrized tests this project did add were themselves not free of authoring bugs (Section 5's sixteen test-authoring failures).

7. Limitations and threats to validity

This study has four limitations worth stating plainly, two of which were raised by Codex during review of an earlier draft of this outline and are folded in directly.

Single project, no replication. One codebase, one author orchestrating every round: there is no inter-rater or inter-project comparison here, and the "24 rounds, no plateau" finding may not generalize to a codebase of different size, domain, or maturity.

Reviewer selection was availability-driven. Which of Codex, Fable, or Claude Opus reviewed a given round depended on tool access at the time (round 24, for instance, used Opus specifically because Codex credits were unavailable), not a designed comparison; this data cannot support a claim that any one of the three is systematically stronger or weaker at this task.

The codebase was a moving target. The system under review was actively growing new feature surface across the 24 rounds (new compounding modes, new calendar and payment-lag parameters, new threshold parameters), not held static. "No defect-discovery plateau across 24 rounds" therefore partly reflects that later rounds had more surface to find defects in, not necessarily that a fixed amount of code would keep yielding defects at the same rate under repeated review. A cleaner version of this study would separate legacy defects (in code untouched by recent rounds) from defects in newly added or recently modified code, normalized by lines changed or features added per round; this article's round table (Table 1) does not make that split,

because no line-level version history was kept from round to round (each round edited the file in place, and the per-round record that exists is the markdown changelog, not a git diff), and reconstructing an approximate split after the fact would itself require a judgment call this article did not have the primary-source data to make rigorously.

Internal consistency is not financial correctness. The large majority of this project's 507 tests, and the large majority of the invariants independent review checked, validate internal or cross-class consistency (a par-rate swap's NPV solving back to zero, `InterestRateSwap` and `FloatingRateNote` agreeing on identical payment-lag inputs, a schedule's cash dates landing where the calendar says they should) rather than agreement with an external reference: a textbook closed-form figure, a second independent implementation, or a real market-quoted instrument's published analytics. Internal consistency is necessary but not sufficient for financial correctness: a systematically wrong day-count convention or discounting basis could, in principle, satisfy every invariant this suite checks while still disagreeing with market convention. A small number of this project's checks are externally validated (the fixed-rate bond example is checked against Jan Röman's published worked figures, and the OIS-versus-LIBOR discounting example against Donald J. Smith's methodology paper), but most of the 507 tests, and most of the 24 rounds' findings, are invariant-only. We flag this distinction here explicitly rather than letting "507 tests pass" imply a stronger correctness claim than the suite's actual composition supports.

8. Future work

The most direct next step, motivated by Section 6's discussion, is adding generative or property-based tests (a library such as Hypothesis, or an equivalent) for the specific invariant classes this project's rounds identified by hand: cash-date invariants across the full calendar/lag/compounding cross-product, par-NPV closure across arbitrary schedule and notional configurations, telescoping under compound versus simple compounding, and calendar-precedence equivalence under an inheritance chain. This is proposed as a complement to continued round-based review, not a replacement for it: every defect found across 24 rounds involved understanding what the code was *trying* to do, not merely whether it satisfied a mechanically-stated invariant, and a review round can propose a new invariant a property test has never been told to check.

A second, narrower line of work is worth naming: this project's own round-24 follow-up fix (restructuring `InterestRateSwap`'s payment-lag calendar inheritance so a dependent collection is derived exactly once, outside every precedence branch, rather than assigned independently inside each one) is a specific instance of a more general pattern: collapsing a duplicated condition down to the single boolean it actually depends on. Whether this generalizes into a standing checklist item for review rounds specifically targeting inheritance or precedence logic is an open question this project's data can motivate but not answer on its own.

Finally, and left open rather than answered: at what round count, if any, does this method's marginal yield actually start to fall, for a codebase of a given size and a given rate of feature growth? Twenty-four rounds of data from a single project is enough to say the plateau had not yet arrived by round 24; it is not enough to say where, or whether, one exists.

9. Conclusion

Across 24 rounds of independent review, a single-file financial pricing library kept yielding genuine, previously undetected defects, with the most concerning pattern in later rounds being defects introduced by the immediately preceding round's own fix rather than defects surviving from untouched legacy code. We report this pattern, alongside two portable lessons it produced (test the invariant a derived value is supposed to guarantee, not the setting that derives it, and calibrate plausibility checks against an input's own scale rather than an absolute cutoff), as this article's central contribution. The round structure itself, independently re-deriving every claim before acting on it in both directions, is offered as a reusable process independent of whether the specific numeric findings generalize beyond this one codebase; the two threats to validity raised during this article's own review, the moving-target confound on "no plateau" and the gap between internal invariants and external correctness on "507 tests pass," are, appropriately, exactly the kind of finding this whole process was built to surface, applied here to itself.

Appendix A: Reproduction index

Every finding in this article traces to a specific reproduction already on record in this project's own files, consistent with the "verify, don't trust" standard the review process itself was built on:

- Round-by-round detail and every headline figure cited in Table 1: `README.md`, sections "Codex review round 1" through "Round 24."
- Round 21's full findings and the three critical schedule-layer bugs: `README.md`, "Round 21."
- Round 22's confirmation of 19/21 round-21 items and the two flaws (dropped-coupon regression, arbitrage-check design flaw): `README.md`, "Round 22"; `CODEX_REQUEST_Duration_Engine_Review_Round21.md`.
- Round 23's three silent wrong-answer bugs and the `BootstrapCurve` threshold rework: `README.md`, "Round 23"; `CODEX_REQUEST_Duration_Engine_Review_Round23.md`.
- Round 24's regression-in-a-fix finding, its three follow-up hardening passes, and the equivalence proof for the final simplification: `README.md`, "Round 24"; `CODEX_REQUEST_Duration_Engine_Review_Round24.md`; `OPUS_REVIEW_Round23.md`.
- Test-suite composition and the sixteen test-authoring failures discussed in Section 5: `tests/test_cash_date_invariants.py`, `tests/test_headline_regression.py`.

- The moving-target confound and the gap between internal invariants and external correctness, both threats to validity from Section 7: folded in from Codex's own review of this article's outline, `RESEARCH_ARTICLE_OUTLINE.md`.

References

1. Röman, J. (2022). *Misunderstanding of Duration calculations for Floaters*. Swedbank Robur.
2. Smith, D. J. (2012). *Valuing Interest Rate Swaps Using OIS Discounting*. Boston University Research Paper No. 2012-11.
3. Hull, J. C. *Options, Futures, and Other Derivatives*. Black-76 option pricing on forward rates (standard textbook treatment).
4. This project's own record: `duration_engine.py`, `README.md`, and 24 rounds of `CODEX_REQUEST_Duration_Engine_Review_RoundN.md` files, `/Users/oluwaseyiawoga/Documents/rl_javascript/duration_engine/`.